

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
ВАРИАНТ 5.

Студент гр. 4343

Маркашов Г.Д.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Задание.

1 задание.

Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст ($T, 1 \leq |T| \leq 1000000$).

Вторая – число $n (1 \leq n \leq 3000)$, каждая следующая из n строк содержит шаблон из набора $P = \{p_1, \dots, p_n\}$ $1 \leq |p_i| \leq 75$

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Выход:

Все вхождения образцов из P в T .

Каждое вхождение образца в текст представить в виде двух чисел – i p

Где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

2 задание.

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с джокером.

В шаблоне встречается специальный символ, именуемый джокером (wildcard), который "совпадает" с любым символом. По заданному содержащему шаблону образцу P необходимо найти все вхождения P в текст T .

Например, образец $ab??c?$ с джокером $?$ встречается дважды в тексте $xabvccbababcsax$.

Символ джокер не входит в алфавит, символы которого используются в T . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида $???$ недопустимы.

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Вход:

Текст (T , $1 \leq |T| \leq 100000$)

Шаблон (P , $1 \leq |P| \leq 40$)

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

Индивидуальное задания.

Вариант 5. Вычислить максимальное количество дуг, исходящих из одной вершины в боре; вырезать из строки поиска все найденные образцы и вывести остаток строки поиска.

Выполнение работы.

1. Задание 1

Программа считывает `text` и кол-во шаблонов `n`.

Строится бор (префиксное дерево) из всех паттернов. Каждая вершина содержит:

- `next` — переходы по символам алфавита $\{A, C, G, T, N\}$ в боре,
- `go` — переходы в автомате (с учётом суффиксных ссылок),
- `sufflink` — суффиксная ссылка,
- `dict_link` — сжатая суффиксная ссылка (на ближайшую терминальную вершину),
- `pattern_ids` — список идентификаторов паттернов, заканчивающихся в данной вершине.

Суффиксные ссылки строятся лениво, то есть вычисляются только при необходимости.

Затем происходит проход по тексту: для каждого символа выполняется переход по `go` и обход по сжатым ссылкам для обнаружения всех вхождений шаблонов.

При нахождении шаблона в результате добавляется пара: позиция начала вхождения и ID шаблона.

Результаты сортируются и выводятся на экран.

2. Задание 2

Программа решает задачу поиска шаблона `p`, содержащего символ-джокер (любой символ, который может быть произвольным символом текста).

Сначала программа разбивает шаблоны на безмасочные строки. Каждая такая строка представляется как индекс начала строки в основной строке и длина строки.

Затем строится автомат для всех таких подстрок. В терминальных вершинах хранятся стартовая позиция и длина.

При проходе по тексту для каждой вершины `i` обновляется массив `C`, где `C[i]` – кол-во найденных подстрок, соответствующих шаблону.

Если `C[i]` равно кол-во подстрок без джокера, то значит все части шаблона совпали с учетом джокера, и позиции `i+1` выводятся как начало вхождений.

3. Основной задание

Программа является расширением программы используемой для решения задачи 1. Задача разделена на 2 подзадачи:

Первая подзадача заключалась в поиске наибольшего кол-ва исходящих дуг в дереве. Решение этой задачи заключалось в одном проходе по всему дереву и подсчету исходящих ребер которые не равны `None` и последующим сравнением с уже найденной максимальным кол-вом.

Вторая подзадача заключалась в вырезании найденных образцов. Решение задачи было следующим: Создается массив `to_delete` длиной с весь текст и заполняется во всех полях `False`. Когда автомат находит образец в

тексте, заканчивающийся на индексе i , мы вычисляли его начало `start_pos`, далее заполняли массив `to_delete` значениями `True` с `start_pos` до $i + 1$. В самом конце собирали строку заново, оставляя только те символы у которых флаг остался `False`.

Код выполнения работы см. в Приложении А.

Сложность алгоритма.

Временная сложность:

N – Длина текста.

K – Кол-во строк-шаблонов

Если текущее состояние автомата является концом одного из шаблонов, оно также может является концом других, более коротких шаблонов, например текст: ааааааа, и шаблоны: а, аа, ааа, аааа. В худшем случае в одной позиции текста могут одновременно завершаться K шаблонов, отсюда следует временная сложность $O(N*K)$

Пространственная сложность:

L – Суммарная длина всех строк-шаблонов

$|\Sigma|$ - Размер алфавита

N – Длина текста

Каждый узел `Vertex` хранит два списка (`next` и `go`) размером $|\Sigma|$. Количество узлов в худшем случае равно $L + 1$ (корень + по одному узлу на каждый символ всех паттернов, если они не пересекаются).

Сложность: $O(L*|\Sigma|)$. При $|\Sigma| = 5$ это $O(L)$.

Список `patterns` занимает $O(L)$.

Список `results` хранит все найденные вхождения, что занимает $O(m)$, где m в худшем случае, когда текст состоит из повторяющихся паттернов, может достигать величины $N * K$, то есть $O(N*K) = O(m)$.

Итоговая пространственная сложность: $O(L*|\Sigma| + m + N)$.

Приложения

Приложение А

5_1.py:

```
class Vertex:
    def __init__(self, id: int, parent, pchar: str, alpha=5):
        self.id = id
        self.next = [None] * alpha
        self.parent = parent
        self.pchar = pchar
        self.sufflink = None
        self.dict_link = None
        self.go = [None] * alpha
        self.pattern_ids = []

    def __str__(self) -> str:
        return f"id: {self.id}"

    def __repr__(self):
        return self.__str__()

def num(c):
    return "ACGTN".index(c)

class Trie:
    def __init__(self, alpha=5, verbose=False):
        self.alpha = alpha
        self.verbose = verbose
        self.vertices = [Vertex(0, None, None, alpha)]
        self.root = self.vertices[0]
        if self.verbose:
            print(f"Создан корневой узел {self.root.id}")

    def size(self):
        return len(self.vertices)

    def last(self):
        return self.vertices[-1]

    def add(self, s: str, pattern_id: int):
        if self.verbose:
            print(f"Добавление шаблона #{pattern_id + 1}: '{s}'")
        v = self.root
        for idx, char in enumerate(s):
            c_idx = num(char)
            if v.next[c_idx] is None:
```

```

        self.vertices.append(Vertex(self.size(), v, char,
self.alpha))
        v.next[c_idx] = self.last()
        if self.verbose:
            print(f"    Создан узел {self.last().id} (символ
'{char}', родитель {v.id})")
            v = v.next[c_idx]
            v.pattern_ids.append(pattern_id)
            if self.verbose:
                print(f"    Узел {v.id} помечен как терминальный для шаблона
#{pattern_id + 1}")

    def get_link(self, v: Vertex) -> Vertex:
        if v.sufflink is None:
            if v == self.root or v.parent == self.root:
                v.sufflink = self.root
                if self.verbose and v != self.root:
                    print(f"    Суффиксная ссылка узла {v.id}
('{v.pchar}') -> корень")
            else:
                if self.verbose:
                    print(f"    Вычисление суффиксной ссылки для узла
{v.id} ('{v.pchar}'))")
                v.sufflink = self.go(self.get_link(v.parent), v.pchar)
                if self.verbose:
                    print(f"    Суффиксная ссылка узла {v.id} -> узел
{v.sufflink.id}")
            return v.sufflink

    def get_dict_link(self, v: Vertex) -> Vertex:
        if v.dict_link is None:
            link = self.get_link(v)
            if link == self.root:
                v.dict_link = self.root
            elif link.pattern_ids:
                v.dict_link = link
                if self.verbose:
                    print(f"    Сжатая ссылка узла {v.id} ->
терминальный узел {link.id}")
            else:
                v.dict_link = self.get_dict_link(link)
        return v.dict_link

    def go(self, v: Vertex, c: str) -> Vertex:
        c_idx = num(c)
        if v.go[c_idx] is None:
            if v.next[c_idx] is not None:
                v.go[c_idx] = v.next[c_idx]
            elif v == self.root:
                v.go[c_idx] = self.root

```

```

        else:
            v.go[c_idx] = self.go(self.get_link(v), c)
        return v.go[c_idx]

if __name__ == "__main__":
    text = input("Введите текст: ")
    n = int(input("Введите количество шаблонов: "))

    print(f"\nТекст: '{text}'")
    print(f"Шаблонов: {n}\n")

    t = Trie(verbose=True)
    pattern_len = [0] * (n + 1)

    print("---- Построение бора ----")
    for i in range(1, n + 1):
        p = input(f"Шаблон #{i}: ")
        pattern_len[i-1] = len(p)
        t.add(p, i-1)

    print("\n--- Поиск вхождений ---")
    results = []
    v = t.root

    for i, char in enumerate(text):
        print(f"\nПозиция {i+1}, символ '{char}':")
        v = t.go(v, char)
        print(f"  Переход в узел {v.id}")

        tmp = v
        while tmp != t.root:
            if tmp.pattern_ids:
                for p_id in tmp.pattern_ids:
                    start_pos = i - pattern_len[p_id] + 2
                    results.append((start_pos, p_id + 1))
                    print(f"  Найдено совпадение: шаблон #{p_id + 1}
(длина {pattern_len[p_id]}) заканчивается в узле {tmp.id}, начальная
позиция {start_pos}")
                tmp = t.get_dict_link(tmp)
            if tmp != t.root and tmp.pattern_ids:
                print(f"  Проверка по сжатой ссылке: узел {tmp.id}")

    print(f"\n--- Результат ---")
    results.sort()
    output = [f"{pos} {p_id}" for pos, p_id in results]
    print("\n".join(output) if output else "Совпадений не найдено")

```

5_2.py:

```
class Vertex:
```

```

def __init__(self, id: int, parent, pchar: str, alpha: int = 5):
    self.id = id
    self.next = [None] * alpha
    self.parent = parent
    self.pchar = pchar
    self.sufflink = None
    self.dict_link = None
    self.go = [None] * alpha
    self.substring_data = [] # (стартовая позиция l_i, длина)

def __str__(self) -> str:
    return f"id: {self.id}"

def __repr__(self):
    return self.__str__()

def num(c):
    return "ACGTN".index(c)

class Trie:
    def __init__(self, alpha: int = 5, verbose: bool = False):
        self.alpha = alpha
        self.verbose = verbose
        self.vertices = [Vertex(0, None, None, self.alpha)]
        self.root = self.vertices[0]

    def size(self) -> int:
        return len(self.vertices)

    def last(self) -> Vertex:
        return self.vertices[-1]

    def add(self, s: str, l_i: int):
        if self.verbose:
            print(f"Добавление подстроки '{s}' (начало в шаблоне:
{l_i}))")
        v = self.root
        for char in s:
            c_idx = num(char)
            if v.next[c_idx] is None:
                self.vertices.append(Vertex(self.size(), v, char,
self.alpha))
            v.next[c_idx] = self.last()
            if self.verbose:
                print(f" Создан узел {self.last().id} для символа
'{char}'")
            v = v.next[c_idx]
        v.substring_data.append((l_i, len(s)))
        if self.verbose:
            print(f" Узел {v.id} хранит данные: ({l_i}, {len(s)})")

```

```

def get_link(self, v: Vertex) -> Vertex:
    if v.sufflink is None:
        if v == self.root or v.parent == self.root:
            v.sufflink = self.root
            if self.verbose and v != self.root:
                print(f" Суффиксная ссылка узла {v.id} ->
корень")
        else:
            if self.verbose:
                print(f" Вычисление суффиксной ссылки для узла
{v.id}")
            v.sufflink = self.go(self.get_link(v.parent), v.pchar)
            if self.verbose:
                print(f" Суффиксная ссылка узла {v.id} -> узел
{v.sufflink.id}")
            return v.sufflink

def get_dict_link(self, v: Vertex) -> Vertex:
    if v.dict_link is None:
        link = self.get_link(v)
        if link == self.root:
            v.dict_link = link
        elif link.substring_data:
            v.dict_link = link
        if self.verbose:
            print(f" Сжатая ссылка узла {v.id} -> узел
{link.id} (имеет данные)")
    else:
        v.dict_link = self.get_dict_link(link)
    return v.dict_link

def go(self, v: Vertex, char: str) -> Vertex:
    c_idx = num(char)
    if v.go[c_idx] is None:
        if v.next[c_idx] is not None:
            v.go[c_idx] = v.next[c_idx]
        elif v == self.root:
            v.go[c_idx] = self.root
        else:
            v.go[c_idx] = self.go(self.get_link(v), char)
    return v.go[c_idx]

if __name__ == "__main__":
    text = input("Текст: ")
    pattern = input("Шаблон: ")
    wildcard = input("Wildcard символ: ")

    print(f"\nТекст: '{text}'")
    print(f"Шаблон: '{pattern}'")

```

```

print(f"Wildcard: '{wildcard}'\n")

t = Trie(verbose=True)

k = 0
cur_part = []

print("--- Разбор шаблона на подстроки ---")
for i, char in enumerate(pattern):
    if char == wildcard:
        if cur_part:
            l_i = i - len(cur_part) + 1
            print(f"Найдена подстрока '{''.join(cur_part)}' (позиции {l_i}-{i} в шаблоне, 1-индексация)")
            t.add("".join(cur_part), l_i)
            k += 1
            cur_part = []
        else:
            cur_part.append(char)

    if cur_part:
        l_i = len(pattern) - len(cur_part) + 1
        print(f"Найдена подстрока '{''.join(cur_part)}' (позиции {l_i}-{len(pattern)} в шаблоне)")
        t.add("".join(cur_part), l_i)
        k += 1

print(f"\nВсего подстрок (k) = {k}")

C = [0] * (len(text) + 2)

print("\n--- Проход по тексту ---")
v = t.root
for i, char in enumerate(text):
    print(f"\nПозиция {i+1} в тексте, символ '{char}':")
    v = t.go(v, char)
    print(f"  Переход в узел {v.id}")

    tmp = v
    while tmp != t.root:
        if tmp.substring_data:
            for l_i, length in tmp.substring_data:
                j = i - length + 2
                start_pos = j - l_i + 1
                if start_pos >= 1 and start_pos + len(pattern) - 1
<= len(text):
                    C[start_pos] += 1
                    print(f"  Найдена подстрока длиной {length} в
тексте на позициях [{j}..{i+1}]")

```

```

        print(f"        Это соответствует {l_i}-й позиции
в шаблоне")

        print(f"        Гипотетическое начало шаблона:
позиция {start_pos}, счетчик -> {C[start_pos]}")
        tmp = t.get_dict_link(tmp)
        if tmp != t.root and tmp.substring_data:
            print(f"        Переход по сжатой ссылке к узлу {tmp.id}")

    print("\n--- Результат ---")
    results = []
    for i in range(1, len(text) + 1):
        if C[i] == k:
            results.append(str(i))
            print(f"Позиция {i}: все {k} подстрок совпали")
        elif C[i] > 0:
            print(f"Позиция {i}: совпало {C[i]} из {k} подстрок")

    if results:
        print(f"\nНайденные стартовые позиции:")
        print("\n".join(results))
    else:
        print("Совпадений не найдено")

```

main.py:

```

class Vertex:
    def __init__(self, id: int, parent, pchar: str, alpha=5):
        self.id = id
        self.next = [None] * alpha
        self.parent = parent
        self.pchar = pchar
        self.sufflink = None
        self.dict_link = None
        self.go = [None] * alpha
        self.pattern_ids = []

    def __str__(self) -> str:
        return f"id: {self.id}"

    def __repr__(self):
        return self.__str__()

def num(c):
    return "ACGTN".index(c)

class Trie:
    def __init__(self, alpha=5, verbose=False):
        self.alpha = alpha
        self.verbose = verbose
        self.vertices = [Vertex(0, None, None, alpha)]

```

```

self.root = self.vertices[0]
if self.verbose:
    print(f"Создан корневой узел {self.root.id}")

def size(self):
    return len(self.vertices)

def last(self):
    return self.vertices[-1]

def add(self, s: str, pattern_id: int):
    if self.verbose:
        print(f"Добавление шаблона #{pattern_id + 1}: '{s}' (длина {len(s)})")
    v = self.root
    for idx, char in enumerate(s):
        c_idx = num(char)
        if v.next[c_idx] is None:
            self.vertices.append(Vertex(self.size(), v, char,
self.alpha))
            v.next[c_idx] = self.last()
            if self.verbose:
                print(f" Создан узел {self.last().id} для символа '{char}'")
            v = v.next[c_idx]
        v.pattern_ids.append(pattern_id)
        if self.verbose:
            print(f" Узел {v.id} отмечен как терминальный для шаблона #{pattern_id + 1}")

def get_link(self, v: Vertex) -> Vertex:
    if v.sufflink is None:
        if v == self.root or v.parent == self.root:
            v.sufflink = self.root
            if self.verbose and v != self.root:
                print(f" Суффиксная ссылка узла {v.id} ->
корень")
        else:
            if self.verbose:
                print(f" Вычисление суффиксной ссылки для узла {v.id}")
            v.sufflink = self.go(self.get_link(v.parent), v.pchar)
            if self.verbose:
                print(f" Суффиксная ссылка узла {v.id} -> {v.sufflink.id}")
            return v.sufflink

def get_dict_link(self, v: Vertex) -> Vertex:
    if v.dict_link is None:
        link = self.get_link(v)

```

```

        if link == self.root:
            v.dict_link = self.root
        elif link.pattern_ids:
            v.dict_link = link
            if self.verbose:
                print(f"      Сжатая ссылка узла {v.id} ->
терминальный узел {link.id}")
            else:
                v.dict_link = self.get_dict_link(link)
        return v.dict_link

def go(self, v: Vertex, c: str) -> Vertex:
    c_idx = num(c)
    if v.go[c_idx] is None:
        if v.next[c_idx] is not None:
            v.go[c_idx] = v.next[c_idx]
        elif v == self.root:
            v.go[c_idx] = self.root
        else:
            v.go[c_idx] = self.go(self.get_link(v), c)
    return v.go[c_idx]

if __name__ == "__main__":
    text = input("Введите текст: ")
    n = int(input("Введите количество шаблонов: "))

    print(f"\nТекст: '{text}' (длина {len(text)})")
    print(f"Количество шаблонов: {n}\n")

    t = Trie(verbose=True)
    pattern_len = [0] * (n + 1)

    print("--- Построение бора ---")
    for i in range(1, n + 1):
        p = input(f"Шаблон #{i}: ")
        pattern_len[i-1] = len(p)
        t.add(p, i-1)

    print("\n--- Анализ бора ---")
    max_edges = 0
    max_vertex_id = -1
    for v in t.vertices:
        cur_edges = sum(1 for child in v.next if child is not None)
        if cur_edges > max_edges:
            max_edges = cur_edges
            max_vertex_id = v.id
    print(f"Максимальное количество исходящих дуг: {max_edges} (узел
{max_vertex_id})")

    print("\n--- Поиск и удаление шаблонов ---")

```

```

to_delete = [False] * len(text)
v = t.root

for i, char in enumerate(text):
    print(f"\nПозиция {i+1}, символ '{char}':")
    v = t.go(v, char)
    print(f"    Переход в узел {v.id}")

    tmp = v
    found_any = False
    while tmp != t.root:
        if tmp.pattern_ids:
            for p_id in tmp.pattern_ids:
                length = pattern_len[p_id]
                start_pos = i - length + 1
                print(f"    Найден шаблон #{p_id + 1} (длина
{length}) на позициях [{start_pos+1}..{i+1}]")
                to_delete[start_pos:i+1] = [True] * length
                found_any = True
            tmp = t.get_dict_link(tmp)

        if not found_any:
            print(f"    Совпадений не найдено")

    print("\n--- Результат ---")
    deleted_count = sum(to_delete)
    print(f"Удалено символов: {deleted_count}")
    print(f"Осталось символов: {len(text) - deleted_count}")

    remaining_text = "".join([text[i] for i in range(len(text)) if not
to_delete[i]])
    print(f"Остаток строки поиска: {remaining_text}")

    if not remaining_text:
        print("Текст полностью удалён")

```